

Systèmes Ouverts et Distribués

Chapitre II :

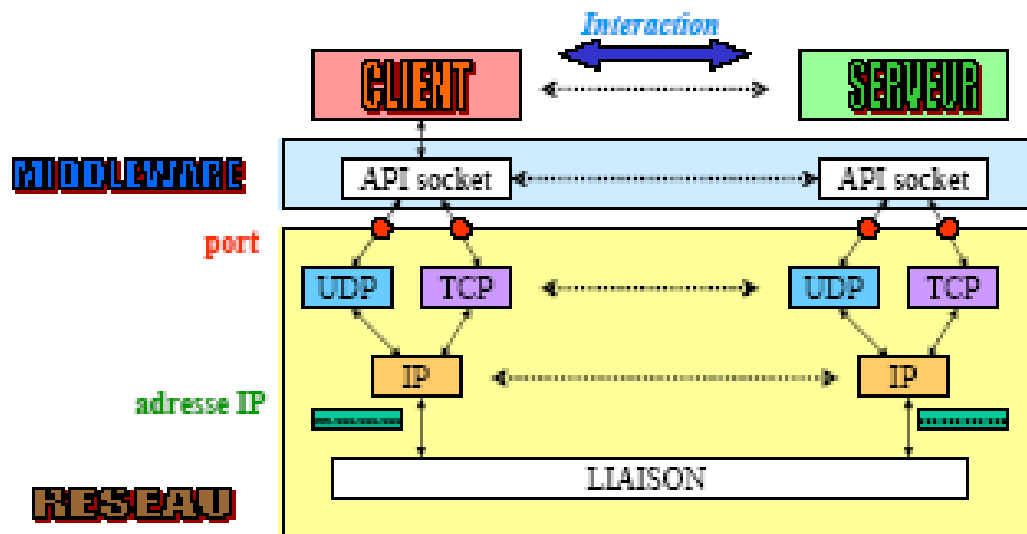
Les sockets TCP/IP

Les SOCKETS : caractéristiques générales (1/3)

- **Une interface**
 - **socket** = interface entre les programmes d'application et les protocoles de communication
 - initialement : UNIX/TCP-IP
 - puis : autres protocoles et systèmes
(*MS-Winsock, Xerox, Apple, mainframe...*)
- **Une abstraction**
 - canal de communication logique accessible par un processus
- **Des primitives**
 - attribuer aux processus un rôle de client ou de serveur
 - échanger des messages protocolaires en rendant transparents les services de communication réseau
 - manipuler des informations de communication
 - contrôler et paramétrer les sockets

Les SOCKETS : caractéristiques générales (2/3)

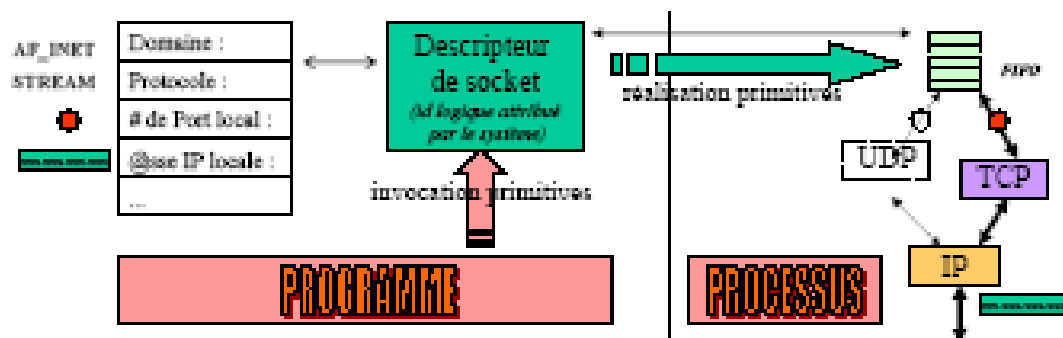
• Illustration



Les SOCKETS : caractéristiques générales (1/3)

• L'abstraction pour le programmeur

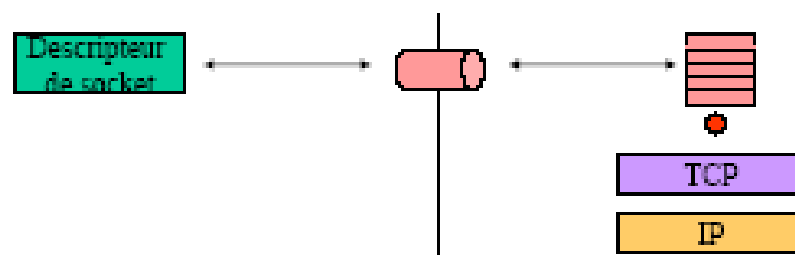
- **vision réseau : association d'application** = quintuplet
 - @use IP locale, # port local,
 - @use IP distance, # port distant
 - mode de communication → protocole (TCP ou UDP)
- **vision système : FIFO** (≠ fichier) + descripteur
- **socket** = extrémité d'association + identificateur logique



Les SOCKETS : Initialisation (1/2)

• Primitive de Création

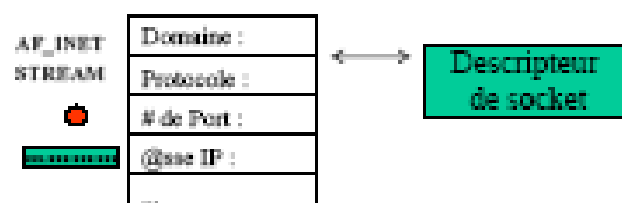
- **primitive socket** : demande au système la création d'une socket devant fonctionner selon un mode de communication (datagramme, connecté) dans un domaine spécifié (AF_INET, AF_UNIX...)
- Le système retourne un descripteur de fichier qui permet d'identifier logiquement ce canal de communication



Les SOCKETS : Initialisation (2/2)

• Primitive d'association d'adresse

- **primitive bind** : demande au système d'associer au descripteur de socket une adresse dans le domaine.
- valeur de l'extrémité de connexion (@sse IP, #port) locale
- Opération non obligatoire chez le client (affectation dynamique d'un numéro de port par le système)



Les SOCKETS : mode connecté (1/10)

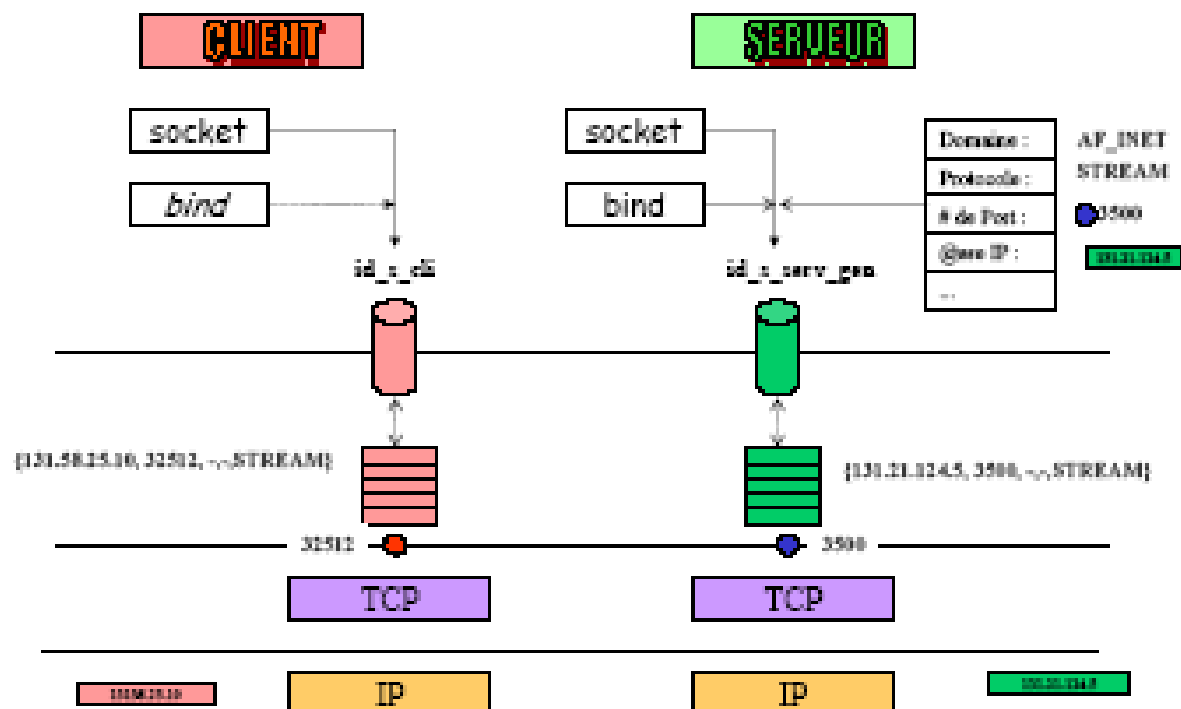
- **Principe**

- flot bidirectionnel d'octets transférés de façon fiable (ni perte, ni duplication) et ordonnée
 - = => service TCP utilisé
- trois phases nécessaires
 - établissement de connexion
 - transfert de données
 - libération de connexion
- au préalable, création et caractérisation des sockets

==>MODE d 'INTERACTION ASYNCHRONE permis

Les SOCKETS : mode connecté (2/10)

- **Création : illustration**



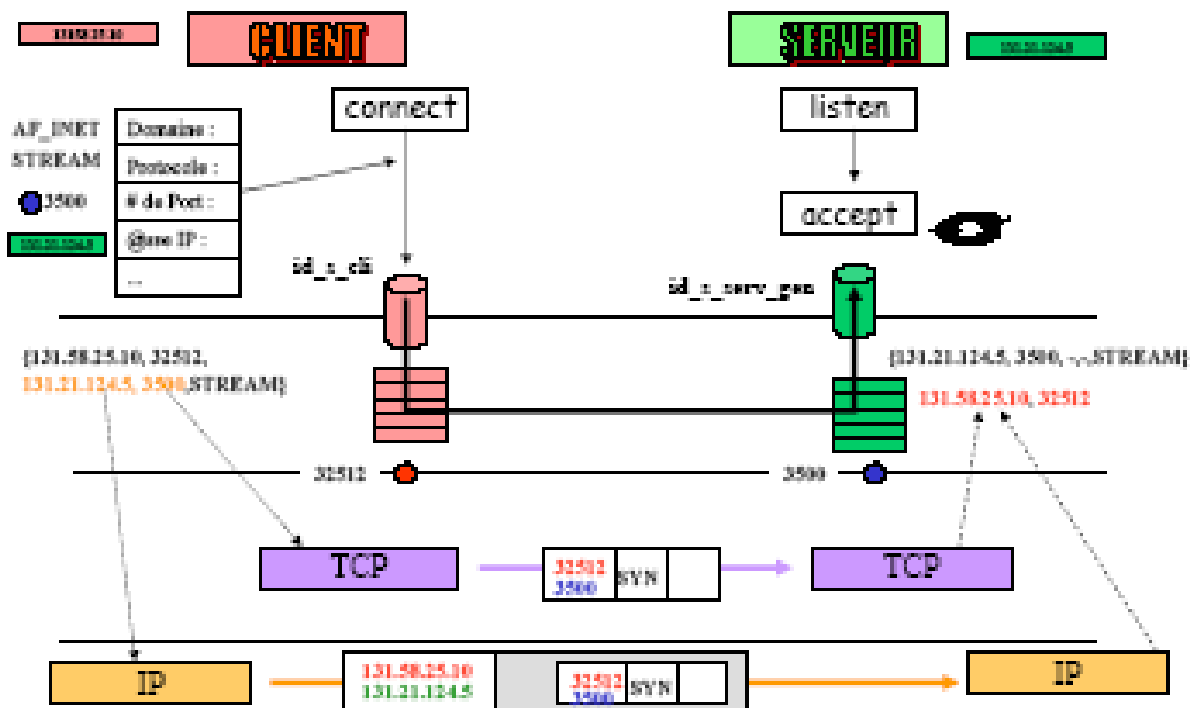
Les SOCKETS : mode connecté (3/10)

• Etablissement de la connexion

- le **client** doit initier l'interaction
 ==> demande d'établissement de connexion
 ==> **primitive connect**
 spécification des caractéristiques de l'extrémité de connexion distante ==> @sse IP, #port socket du serveur
- le **serveur** doit être capable de considérer des demandes de connexion ==> écoute sur une socket « générique »
 ==> **primitive listen** : le processus serveur est connectable sur la socket et se bloque jusqu'à l'arrivée d'une demande de connexion entrante (*ouverture passive*)
- Il doit ensuite accepter une demande et différencier la connexion ainsi établie
 ==> **primitive accept** + création d'une nouvelle socket dédiée à cette connexion avec un descripteur logique spécifique affecté par le système : ce descripteur est utilisé pour la suite des opérations sur cette connexion.

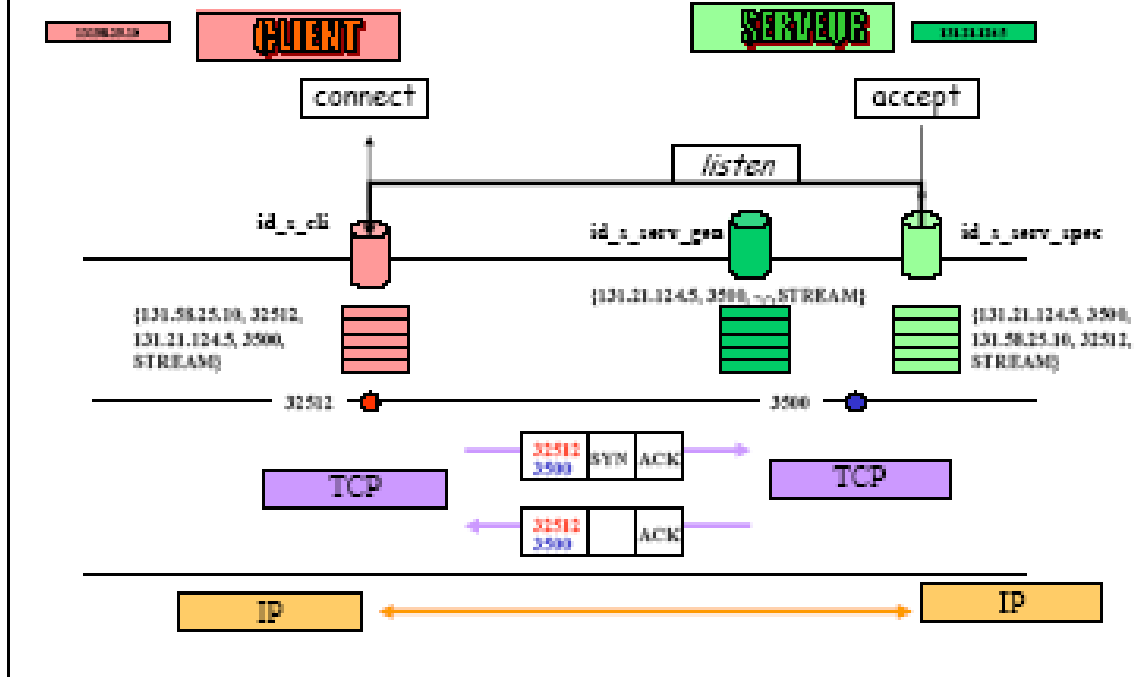
Les SOCKETS : mode connecté (4/10)

• Etablissement de connexion : illustration



Les SOCKETS : mode connecté (5/10)

• Etablissement de connexion : illustration



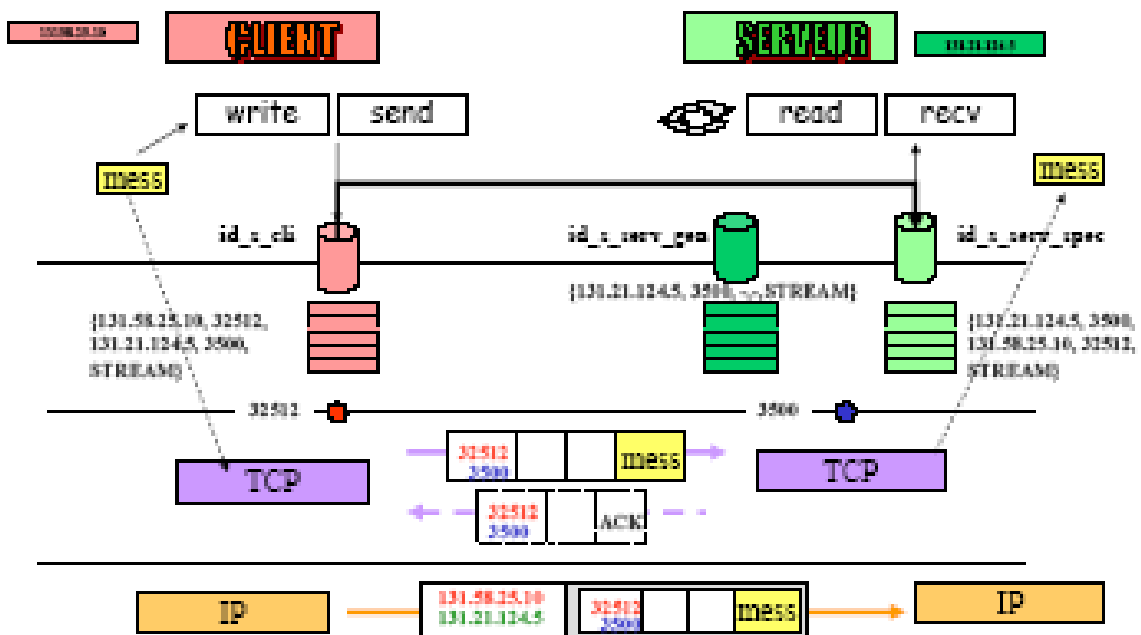
Les SOCKETS : mode connecté (6/10)

• Transfert de données

- Supporter les échanges protocolaires applicatifs
- Pour une émission (≡ écriture fichier)
== > **primitives send** ou **write**
- Pour une réception (≡ lecture fichier)
== > **primitives recv** ou **read**
bloquantes tant que rien n'est à prélever
- sockets utilisées :
 - pour le **client** : sur sa socket
 - pour le **serveur** : sur la socket spécifique à cette connexion

Les SOCKETS : mode connecté (7/10)

- Transfert de données : illustration



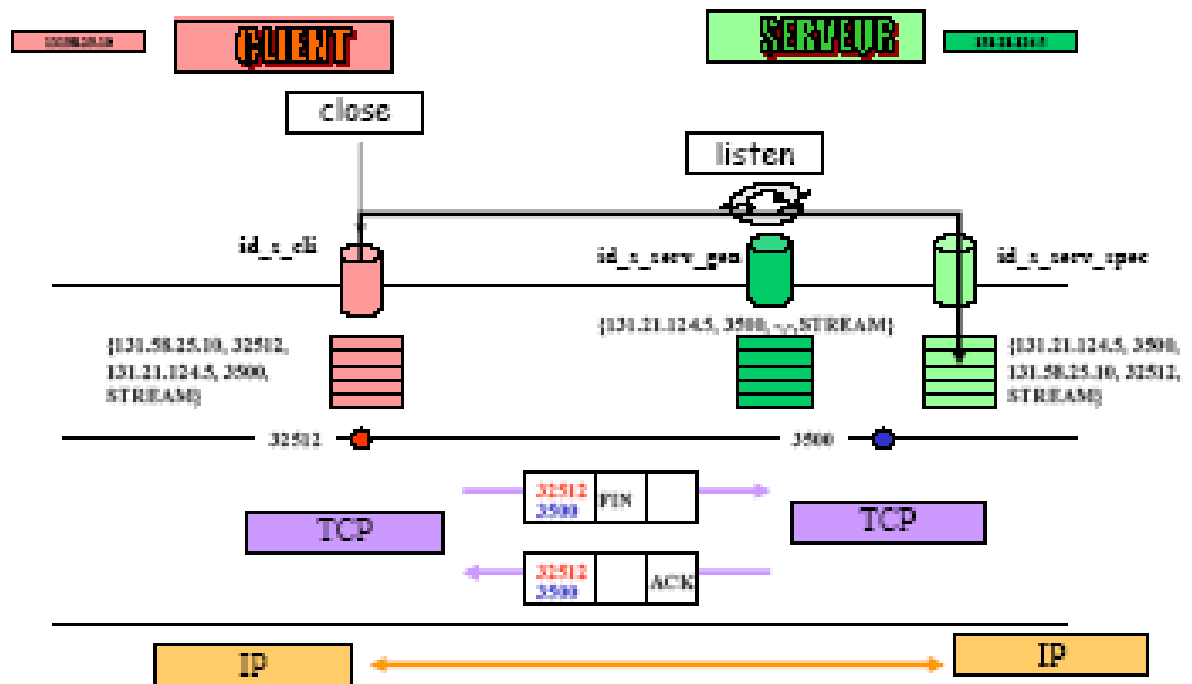
Les SOCKETS : mode connecté (8/10)

- Libération de connexion

- Client et serveur doivent « libérer » chacun leur extrémité de connexion
 - = > **primitive close** (= fermeture fichier)

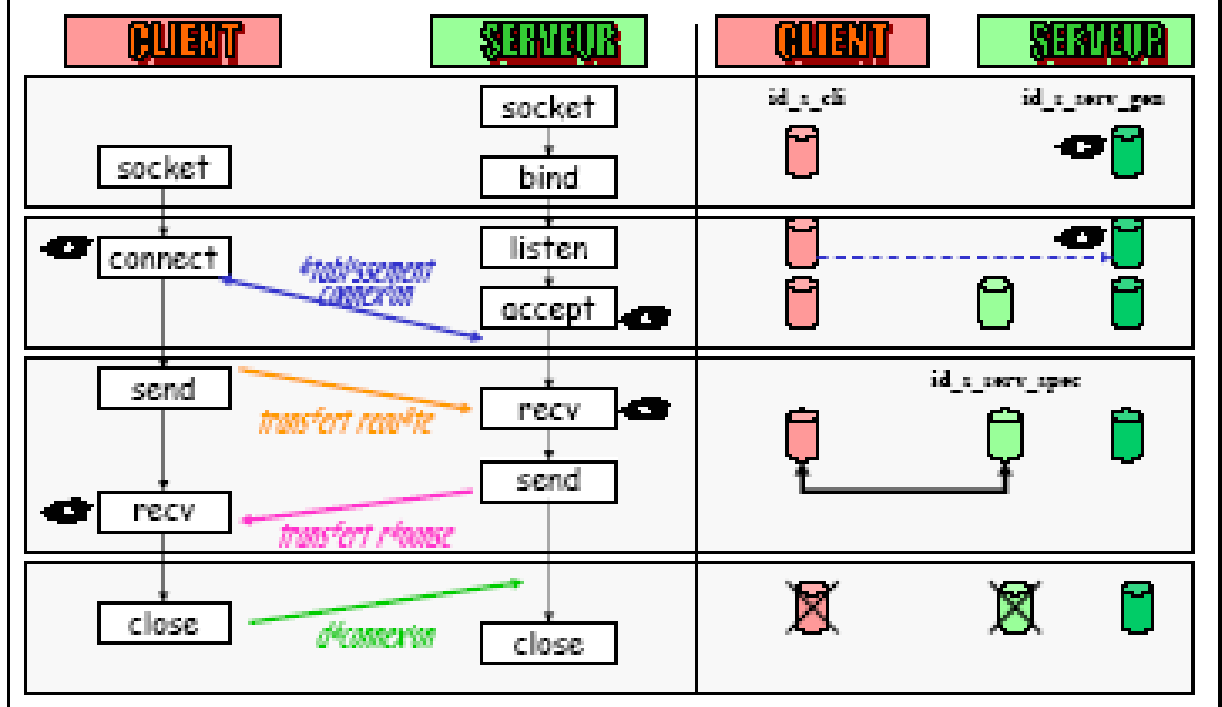
Les SOCKETS : mode connecté (9/10)

• Libération de connexion : illustration



Les SOCKETS : mode connecté (10/10)

• Récapitulatif



Les SOCKETS : mode datagramme (1/4)

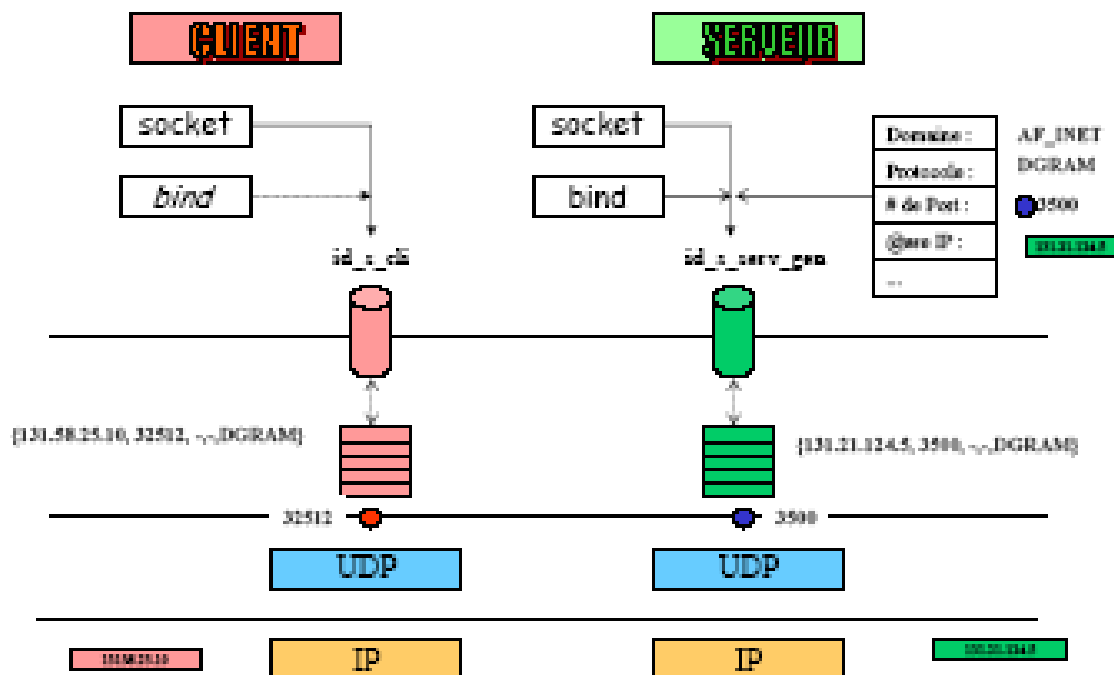
• Principe

- Une seule phase : transfert de données, non fiable (UDP)
- Le **client** initie l'interaction
 - = => envoi de message protocolaire
 - = => **primitive sendto**
 - spécification des caractéristiques de l'extrémité de connexion distante => @sse IP, #port socket du serveur
- Le **serveur** attend des messages sur sa socket
 - = => réception de message protocolaire
 - = => **primitive recvfrom**
 - récupération des caractéristiques de l'extrémité de connexion distante => @sse IP, #port socket du client
- approche symétrique pour le transfert de la réponse

= => **MODE d'INTERACTION SYNCHRONE**

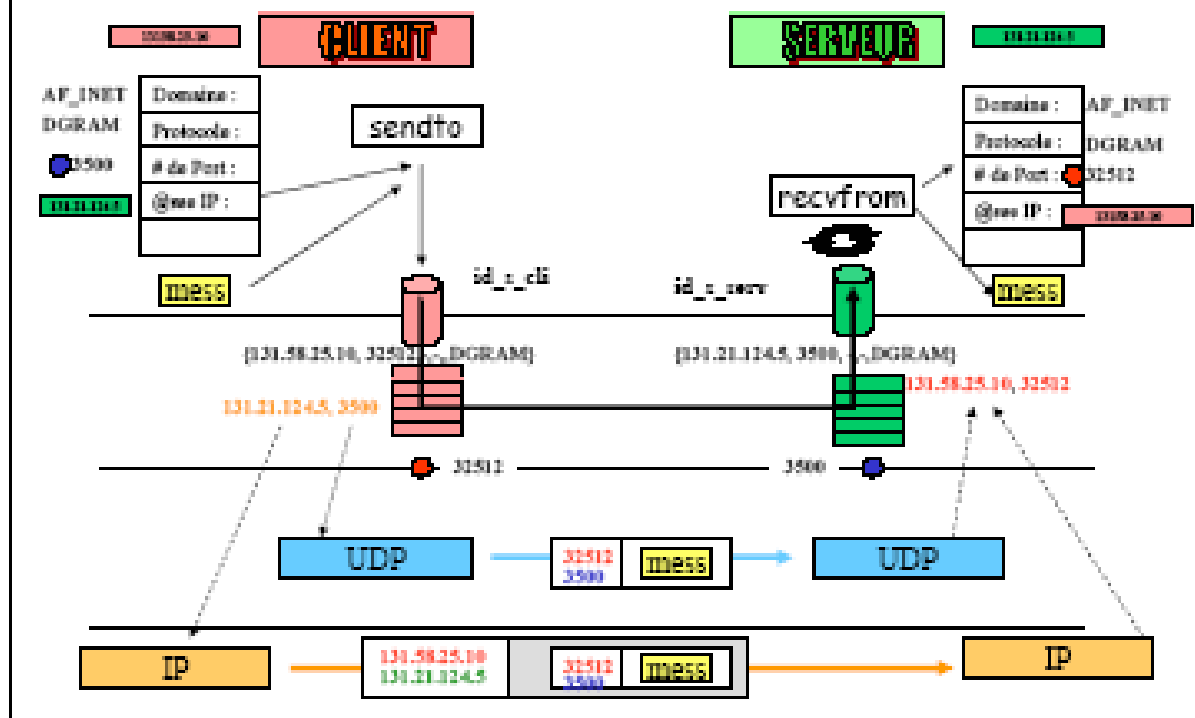
Les SOCKETS : mode datagramme (2/4)

• Création : illustration



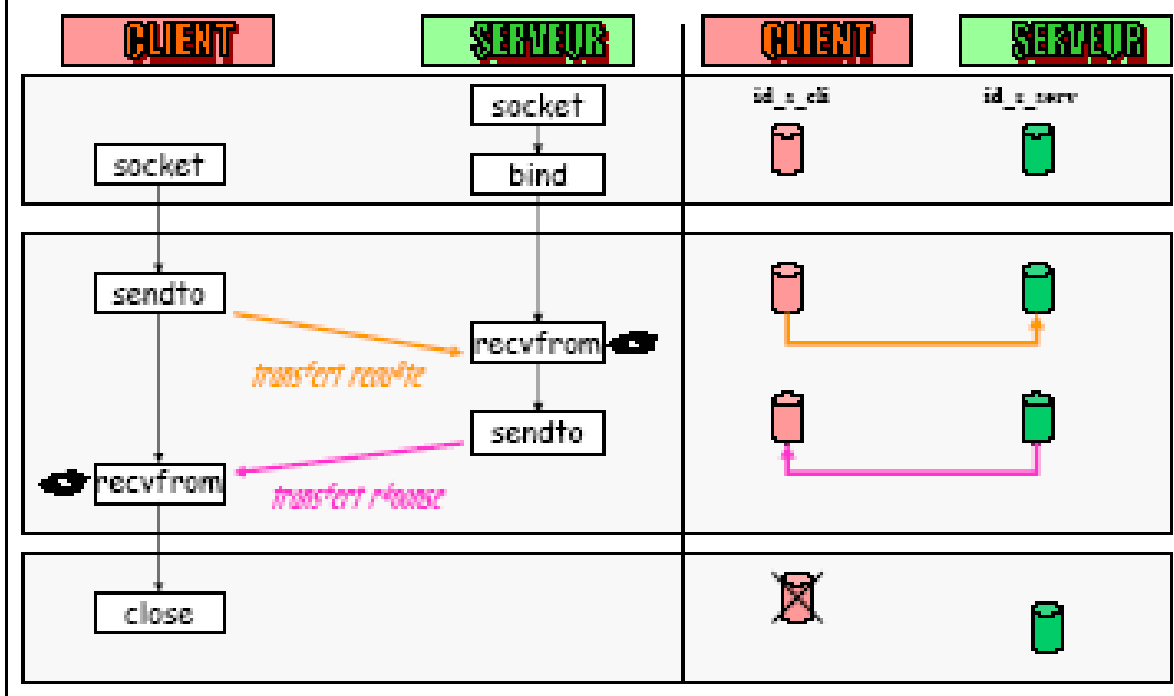
Les SOCKETS : mode datagramme (3/4)

• Transfert de données



Les SOCKETS : mode datagramme (4/4)

• Récapitulatif



ARCHITECTURE des SERVEURS (1/6)

- **Serveur Itératif : un client à la fois...**

- un processus serveur offre via une socket une connectivité sur le réseau
- entre dans un indéfiniment dans un processus d'attente de requêtes clientes
- lorsqu'une requête arrive sur la socket, le processus déclenche le traitement de la requête puis émet le résultat vers le client

==> CLIENTS TRAITÉS en SÉQUENCE, PROCESSUS UNIQUE

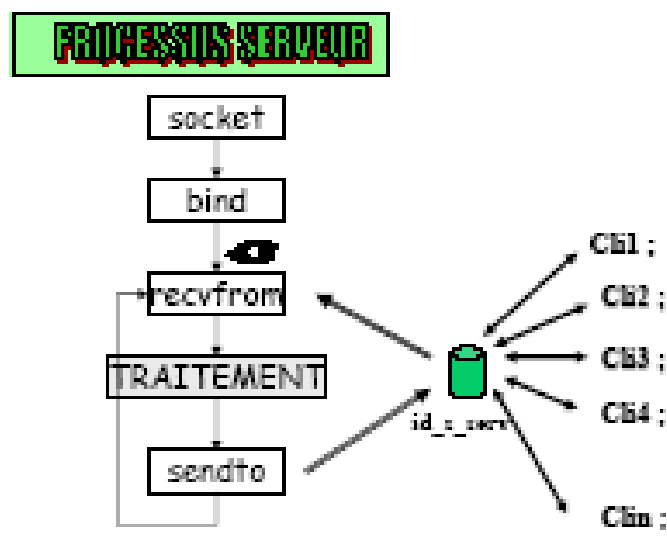
- **Serveur Concurrent : plusieurs clients à la fois...**

- un processus serveur offre via une socket une connectivité sur le réseau
- répétitivement : réceptionne une initiative d'interaction d'un client, offre une nouvelle socket et crée un processus secondaire chargé de traiter l'interaction courante

**==> CLIENTS TRAITÉS en PARALLELE,
MULTIPLES PROCESSUS créés dynamiquement**

ARCHITECTURE des SERVEURS (2/6)

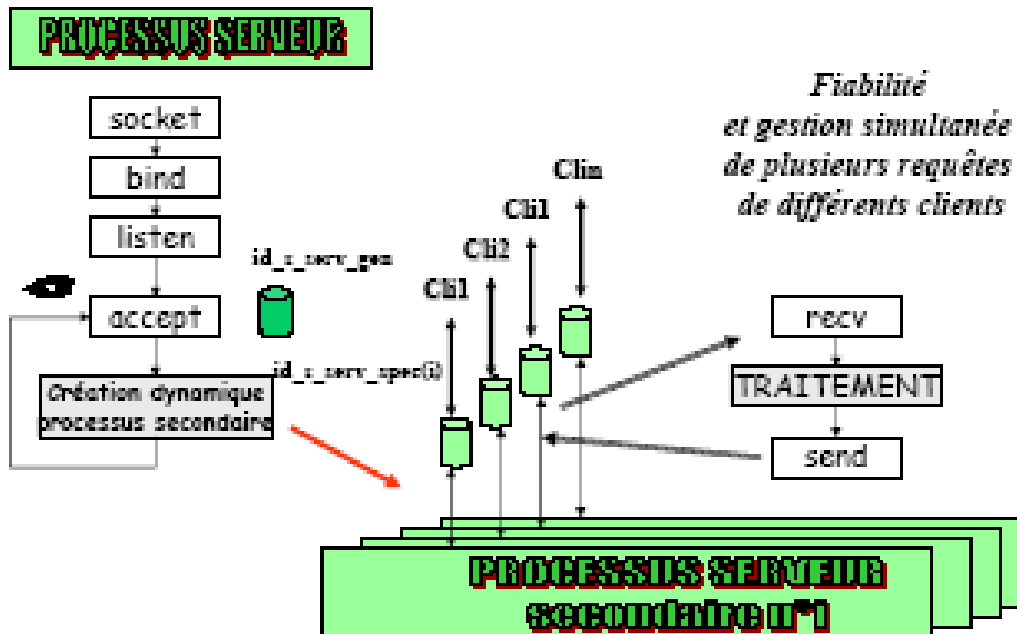
- **Exemple : Serveur Itératif en mode datagramme**



*Architecture adaptée
aux services nécessitant
très peu de traitement
par requête
(ex: TIME)*

ARCHITECTURE des SERVEURS (3/6)

• Exemple : Serveur Concurrent en mode connecté



ARCHITECTURE des SERVEURS (4/6)

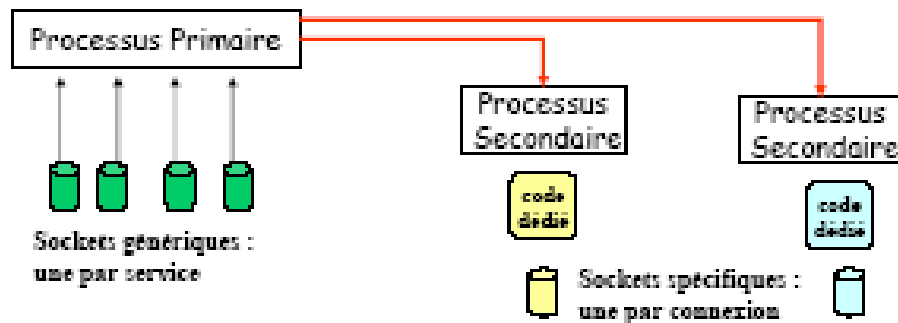
• Serveur Multi-Protocoles

- Le même service est offert simultanément sur plusieurs protocoles de communication
exemple : DAYTIME port 13 sur UDP et TCP
- Satisfaire les clients qui ont des exigences différentes de transfert (fiabilité)
- Deux approches d'implémentation :
 - un serveur par protocole, l'un servant les requêtes TCP, l'autre les requêtes UDP
==> plus aisément contrôlable (accès UDP interdits sur système)
 - un seul et même serveur capable de servir à la fois les requêtes TCP et UDP
==> non duplication des ressources, cohérence dans la gestion du service
==> implémentation possible
 - en mode itératif
 - en mode concurrent

ARCHITECTURE des SERVEURS (5/6)

• Serveur Multi-Services

- Un même serveur offre plusieurs services
- Diminuer le nombre de processus, de ressources consommées
- Principe général d'architecture :



- Serveur multi-services et multi-protocoles :
cas du daemon `inetd` « super serveur Internet » sous UNIX
==> interface de configuration (`/etc/services`)

ACHITECTURE des SERVEURS (6/6)

• Multiplexage d'Entrée/Sortie

- Pouvoir « surveiller » plusieurs descripteurs d'entrée/sortie, en particulier plusieurs sockets, en vue d'y opérer des réceptions et émissions
==> **primitive select**
- Utile pour les serveurs « multi » ...

Les SOCKETS : compléments (1/2)

- **Primitives de gestion d'informations**

- gestion d'adresses de sockets locales ou distantes :
==> **primitives** `getsockname`, `getpeername`
- gestion de noms d'ordinateurs :
==> **primitives** `gethostbyname`, `gethostbyaddr`, `sethostname`
- gestion de noms de domaines :
==> **primitive** `getdomainname`
- gestion de réseau :
==> **primitives** `getnetbyname`, `getnetbyaddr`
- gestion de protocoles :
==> **primitives** `getprotobyname`, `getprobynumber`
- gestion de services :
==> **primitives** `getservbyname`

Les SOCKETS : compléments (2/2)

- **Primitives de représentation et de conversion**

- représentation normalisée TCP/IP pour les entiers utilisés dans les protocoles (ex : #port) appelée *Network Byte Order*.
conversion représentation machine locale/NBO :
==> **primitives** `htonl`, `htons`, `ntohl`, `ntohs`
- manipulation d'adresses IP : conversion 32 bits/notation décimale pointée :
==> **primitives** `inet_addr`, `inet_network`

- **Primitive de contrôle des sockets**

- récupération d'informations caractérisant la socket
 - configuration de valeurs de temporisations, allocation de mémoire tampon, autorisation données urgentes
- ==> **primitive** `getsockopt`